

μOS++ III Architecture Analysis

Single-Core (`xpack-development`) vs. Multi-Core SMP (`smp`)

Author: Antigravity AI Engineering Team | Standards: ISO C++17 / C++20 | Date: September 2026

Contents

1	Executive Overview & Architectural Philosophy	3
1.1	The One-Way Dependency Topology	3
2	Locking, Synchronization & Concurrency Model	4
2.1	The Single-Core Paradigm: Interrupt Masking	4
2.2	The SMP Multi-Core Paradigm: Why Masking Fails	4
2.3	The Recursive Kernel Spinlock (<code>_smp_klock</code>)	4
2.3.1	The Two-Phase Protocol	5
2.4	Scheduler Critical Section vs. Interrupts Critical Section	5
2.4.1	1. Scheduler Critical Section (<code>scheduler::critical_section</code>)	6
2.4.2	2. Interrupts Critical Section (<code>interrupts::critical_section</code>)	7
2.5	Fine-Grained Locking: The Timer Leaf Lock (<code>_smp_tlock</code>)	7
2.6	Hardware Locks & ISA Primitives	7
3	Context Switching & The Deferred Publish Protocol	8
3.1	The Cross-Core Stack Corruption Hazard	8
3.2	The Deferred Publish / Claim Mechanism	8
3.3	Floating Point Unit (FPU) Context & Lazy Stacking	9
4	Scheduler Architecture & Thread Lifecycle	9
4.1	Scheduler Data Structures	9
4.2	Thread CPU Affinity & The Multi-Core Ready-List Picker	9
4.3	Thread Lifecycle, Teardown & <code>join()</code> Synchronization	10
4.3.1	1. Arbitration via <code>state::destroying = 7</code>	10
4.3.2	2. SMP <code>join()</code> Spin-Wait	10
4.4	Inter-Processor Interrupts (IPI)	10
5	Intrusive Lists & Core Data Structures	10
5.1	Why Intrusive Containers in Embedded RTOS Design	10
5.2	Static Destruction & The Clean <code>_Exit()</code> Bypass	11
6	C++ Object-Oriented Solutions, Idioms & Standards	11
7	Architecture-Specific Ports: Deep Technical Breakdown	12
7.1	POSIX Native Host Port (<code>micro-os-plus-iii-posix-arch</code>)	12
7.2	ARM Cortex-M Port (<code>micro-os-plus-iii-cortexm</code>)	12
7.2.1	1. Raspberry Pi Pico 2 (RP2350 Dual-Core Cortex-M33)	12
7.2.2	2. ARM Generic Dual-Core (SSE-200)	12
7.3	ARMv7-A / AArch32 Port (<code>micro-os-plus-iii-aarch32</code>)	12
7.4	ARMv8-A / AArch64 Port (<code>micro-os-plus-iii-aarch64</code>)	12
7.5	RISC-V Port (<code>micro-os-plus-iii-riscv</code>)	13
8	Hardening & Upstream Integration Strategy	13
8.1	Summary of Fixed Defects	13
8.2	The Inviolable Migration Rules	13

8.3 Automated Step Verification Script (scripts/verify-step.sh)	13
8.4 Comprehensive 30-Step Execution Checklist	14
9 Holistic Architecture in Action: Complete Logical Multi-Core Example	16
9.1 Architectural Scenario & Execution Model	16
9.2 Complete C++ Application Code	16
9.3 Step-by-Step State Transition Walkthrough	18
9.4 Single-Core vs. Multi-Core SMP Behavioral Comparison	19

1 Executive Overview & Architectural Philosophy

The **μOS++ III** operating system is an object-oriented Real-Time Operating System designed for embedded micro-controllers and high-performance multi-core SoCs. It implements CMSIS-RTOS v1/v2, POSIX threads, and ISO C++ standard library synchronization primitives.

Within the repository ecosystem, two active branches represent fundamentally distinct runtime models:

Dimension	xpack-development (Single-Core)	smp (Multi-Core SMP)
Execution Topology	Uniprocessor (<code>OS_NCPU = 1</code>).	N Symmetric Cores ($N \geq 1$, e.g. 2, 3, 4, 8).
Scheduler State	Scalar <code>current_thread_</code> , single <code>os_idle_thread_</code> .	Per-CPU arrays <code>current_thread_[OS_NCPU]</code> , <code>os_idle_thread_core[OS_NCPU]</code> .
Mutual Exclusion	Local CPU interrupt disable (<code>CPSID</code> , <code>PRIMASK</code>).	Two-Phase: Per-CPU interrupt masking + Recursive Kernel Spinlock (<code>_smp_klock</code>).
Context Switch	Immediate stack pointer reassignment.	Two-Phase Deferred Publish / Claim Protocol preventing cross-core stack clobbering.
Thread Affinity	None (all threads execute on the sole CPU).	32-bit affinity mask (<code>th_cpu_affinity</code>), affinity-aware ready list picker.
Thread Migration	Irrelevant.	Free migration across cores, strict ban on native TLS caching.
Architecture Ports	Monolithic uniprocessor ports.	Decoupled multi-repo ports (<code>posix-arch</code> , <code>cortexm</code> , <code>aarch32</code> , <code>aarch64</code> , <code>riscv</code>).

1.1 The One-Way Dependency Topology

To maintain clean mergeability with upstream while enabling clean platform extensions, dependencies run strictly in one direction:

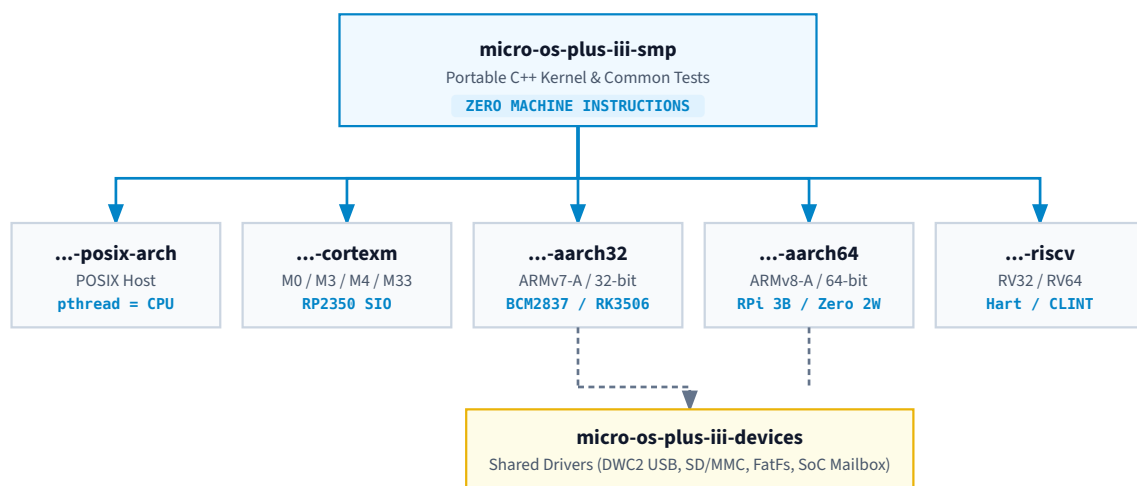


Figure 1: Figure 1: μOS++ III Multi-Repository Ecosystem and One-Way Dependency Model

1. **The Kernel (`micro-os-plus-iii-smp`)**: Contains zero machine assembly instructions. Implements portable scheduling, synchronization objects, and timers.

2. **Architecture Ports** (`micro-os-plus-iii-<arch>`): Supply machine word sizes, interrupt masking instructions, CPU ID extraction, hardware timers, and context switch assembly.
3. **Device Layer** (`micro-os-plus-iii-devices`): Shared drivers (DWC2 USB controller, SD card, FlatFS, SoC mailboxes) identical across 32-bit and 64-bit architectures.

2 Locking, Synchronization & Concurrency Model

2.1 The Single-Core Paradigm: Interrupt Masking

In `xpack-development`, only one instruction stream executes at any moment. Concurrency hazards arise solely from asynchronous preemption (e.g. SysTick timer interrupt). Disabling local interrupts (`CPSID i` or `PRIMASK`) guarantees absolute atomicity across the entire system.

```
// Single-Core Critical Section:
inline rtos::interrupts::state_t critical_section::enter (void) {
    return port::interrupts::critical_section::enter (); // e.g. cpsid i / PRIMASK
}
inline void critical_section::exit (rtos::interrupts::state_t state) {
    port::interrupts::critical_section::exit (state);    // e.g. cpsie i / PRIMASK
}
```

2.2 The SMP Multi-Core Paradigm: Why Masking Fails

On an SMP system, disabling interrupts on Core 0 has **zero effect** on Core 1 or Core 2. While Core 0 modifies an intrusive list, Core 1 can simultaneously execute, dereference the same pointers, and write to the same memory addresses, causing immediate data corruption.

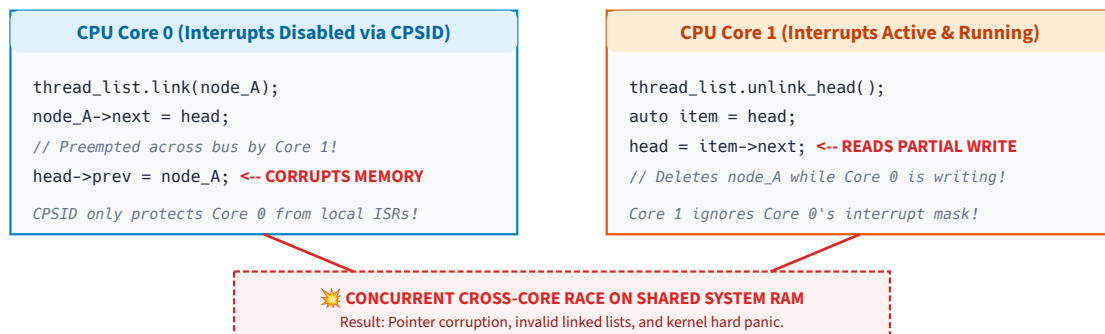


Figure 2: Multi-Core Concurrency Hazard — Why Local Interrupt Masking Fails on SMP Silicon



The SMP Concurrency Hazard

Interrupt masking alone on multi-core silicon does not serialize memory accesses across different CPU cores. Multi-core synchronization requires hardware-enforced memory bus arbitration via atomic operations and spinlocks.

2.3 The Recursive Kernel Spinlock (`_smp_klock`)

To guarantee mutual exclusion across all cores without deadlocking nested critical sections, the `smp` branch introduces `smp_klock_t`:

```
typedef struct {
    volatile uint32_t lock;    // 0 = unlocked, 1 = locked
    volatile uint32_t owner;  // CPU ID of owner, or SMP_NO_OWNER (0xFFFFFFFF)
    volatile uint32_t depth;  // Nesting depth for recursive critical sections
} smp_klock_t;
```

```
inline void _smp_klock_raw_acquire (void) {
    while (__atomic_exchange_n (&smp_klock.lock, 1u, __ATOMIC_ACQUIRE) != 0u) {
#ifdef __x86_64__ || defined(__i386__)
        __builtin_ia32_pause (); // PAUSE: relaxes core pipeline
#elif defined(__aarch64__) || defined(__arm__)
        __asm__ volatile ("yield" ::: "memory"); // YIELD: yields interconnect priority
#elif defined(__riscv)
        __asm__ volatile ("pause" ::: "memory");
#endif
    }
}
```

2.3.1 The Two-Phase Protocol

Entering a critical section follows two discrete phases:

1. **Phase 1:** Mask local CPU interrupts (preventing local ISR preemption).
2. **Phase 2:** Acquire the cross-core recursive spinlock.

```
inline rtos::interrupts::state_t critical_section::enter (void) {
    rtos::interrupts::state_t prior_irq = port_mask_local_interrupts ();
    const unsigned cpu = scheduler::port_cpu_id_inline ();

    if (scheduler::_smp_klock.owner != cpu) {
        scheduler::_smp_klock_raw_acquire ();
        scheduler::_smp_klock.owner = cpu;
    }
    scheduler::_smp_klock.depth++;
    return prior_irq;
}

inline void critical_section::exit (rtos::interrupts::state_t prior_irq) {
    const unsigned cpu = scheduler::port_cpu_id_inline ();
    if (scheduler::_smp_klock.owner == cpu && scheduler::_smp_klock.depth > 0) {
        scheduler::_smp_klock.depth--;
        if (scheduler::_smp_klock.depth == 0) {
            // CRITICAL ORDER: Clear owner FIRST, release lock word LAST
            scheduler::_smp_klock.owner = SMP_NO_OWNER;
            __atomic_store_n (&scheduler::_smp_klock.lock, 0u, __ATOMIC_RELEASE);
        }
    }
    port_restore_local_interrupts (prior_irq);
}
```

Memory Ordering Guarantee

Clearing owner = SMP_NO_OWNER before releasing lock = 0 with __ATOMIC_RELEASE ensures that another core acquiring the lock immediately sees valid owner state, preventing metadata corruption.

2.4 Scheduler Critical Section vs. Interrupts Critical Section

In real-time embedded software design, conflating scheduler locking with interrupt masking is one of the most common causes of system failure and priority inversion. µOS++ provides two distinct synchronization abstractions:

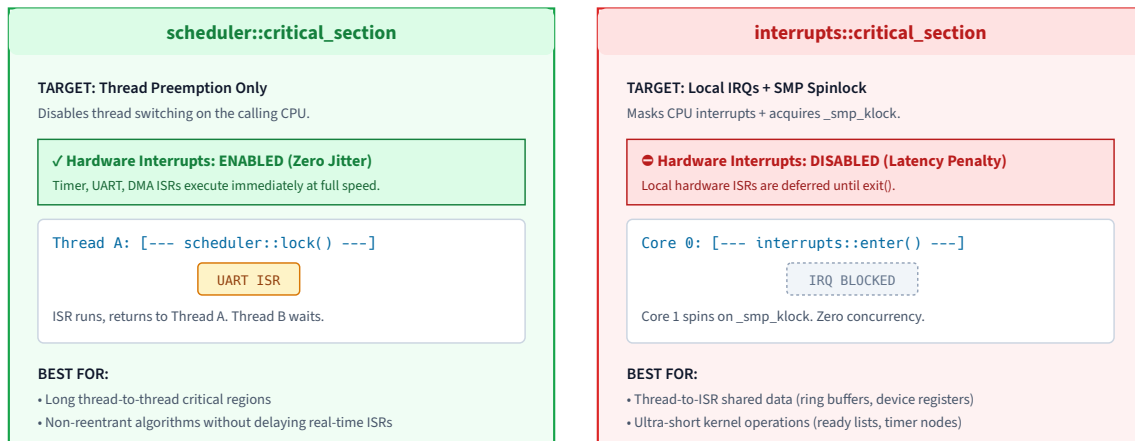


Figure 3: Figure 3: Scheduler Critical Section vs. Interrupts Critical Section Execution Model

Property	<code>scheduler::critical_section</code>	<code>interrupts::critical_section</code>
Target Scope	Thread-to-Thread Preemption Only.	Thread-to-ISR and Cross-Core Hardware Bus.
Hardware IRQs	ENABLED (Zero interrupt jitter/latency).	DISABLED / MASKED on the calling core.
SMP Locking	Sets <code>lock_state[cpu] = locked</code> .	Acquires global <code>_smp_klock</code> recursively.
Allowed Context	Thread Mode ONLY (<code>!in_handler_mode()</code>).	Thread Mode AND Handler Mode (ISRs).
Blocking Calls	FORBIDDEN (Throws <code>EPERM</code> assert).	FORBIDDEN (Deadlocks system).
Duration	Can be long (e.g. non-reentrant algorithms).	Must be ultra-short (a few clock cycles).

2.4.1 1. Scheduler Critical Section (`scheduler::critical_section`)

Purpose & Guarantees: When a thread enters `scheduler::critical_section`, it prevents the RTOS scheduler from switching context away to another thread on the current CPU core. However, **hardware interrupts (SysTick, UART, DMA, CAN, Ethernet) remain fully enabled**.

```
// RAII Usage of Scheduler Critical Section:
{
    os::rtos::scheduler::critical_section scs; // Scheduler locked on this core
    // Long-running non-reentrant user calculation or legacy C library function
    process_complex_buffer(shared_data);
    // Hardware interrupts fire seamlessly without missing data packets!
} // Exiting automatically unlocks scheduler and triggers deferred reschedule if pending
```

SMP Implementation Mechanics: In `os-core.cpp`, `scheduler::locked(state)` guards against thread migration while modifying `lock_state[cpu]`:

```
state_t locked (state_t state) {
    os_assert_throw (!interrupts::in_handler_mode (), EPERM);
    #if defined(OS_USE_SMP_SCHEDULER)
        // Mask IRQs temporarily BEFORE reading port_cpu_id() to prevent migration
        uint32_t pri = __get_PRIMASK ();
        __asm__ volatile ("cpsid i" ::: "memory");
        unsigned cpu = port_cpu_id ();
```

```

state_t tmp = lock_state[cpu];
if (state != tmp) {
    if (state == state::locked) {
        port_set_lock ();
        lock_primask[cpu] = pri;
        lock_state[cpu] = state;
    } else {
        lock_state[cpu] = state;
        port_put_lock (lock_primask[cpu]);
    }
} else {
    __set_PRIMASK (pri);
}
return tmp;
#endif
}

```

Strict Rule — No Blocking Operations: A thread holding `scheduler::critical_section` must **never** execute blocking primitives (`sleep_for()`, `semaphore::wait()`, `mutex::lock()`, `mqueue::receive()`). All kernel blocking paths enforce this via:

```
os_assert_err (!scheduler::locked (), EPERM);
```

If a thread blocked while the scheduler was locked, the scheduler would be unable to perform a context switch to idle or another thread, hanging the CPU core permanently.

2.4.2 2. Interrupts Critical Section (`interrupts::critical_section`)

Purpose & Guarantees: Disables all maskable hardware interrupts on the calling core and takes the global `_smp_klock`. It provides total, atomic mutual exclusion across the entire SoC.

When to Use:

- Protecting data shared between a **Thread and an ISR** (e.g. a hardware FIFO buffer read by a thread and written by an interrupt).
- Manipulating low-level kernel intrusive structures (`ready_list_`, `timer_node`, memory pool headers).

Why Duration Must Be Minimized: Because hardware interrupts are masked, any ISR raised while in `interrupts::critical_section` is delayed until exit. Prolonged interrupt critical sections cause **dropped hardware packets, UART buffer overruns, and motor control jitter**.

2.5 Fine-Grained Locking: The Timer Leaf Lock (`_smp_tlock`)

Holding the global `_smp_klock` during high-frequency hardware timer updates creates severe bus contention. The `smp` branch implements a fine-grained **Timer Leaf Lock** (`_smp_tlock`):

- `port_tmr_lock()` / `port_tmr_unlock()` lock only the timer queue during arming/disarming.
- Thread scheduling and list reordering continue uninterrupted on other cores.

2.6 Hardware Locks & ISA Primitives

Architecture	Primitive	Implementation Details
AArch64	<code>LDAXR</code> / <code>STLXR</code>	Load-Acquire / Store-Release Exclusive synchronized by ARMv8 Point of Coherency (PoC).
AArch32	<code>LDREX</code> / <code>STREX</code>	Load/Store Exclusive with data barrier <code>DMB ISH</code> (Inner Shareable).
RP2350 (M33)	SIO Spinlock 0	Hardware Spinlock Registers (<code>0xD0000100</code>). RP2350 has no global exclusive monitor across cores; SIO hardware spinlocks provide single-cycle atomic test-and-set.

POSIX Host	<code>__atomic_exchange_n</code>	Maps to <code>LOCK XCHG</code> on x86-64 or <code>LDXR / STXR</code> on host ARM.
RISC-V	<code>AMO.SWAP.W.AQ</code>	Atomic Memory Operation with acquire/release annotation.

3 Context Switching & The Deferred Publish Protocol

3.1 The Cross-Core Stack Corruption Hazard

In a uniprocessor RTOS, context switching is immediate. On SMP, a dangerous hazard exists:

1. Core 0 yields Thread A and puts Thread A back onto the ready list.
2. Core 1 searches the ready list, finds Thread A, and immediately dispatches it.
3. Core 1 loads Thread A's stack pointer and starts executing.
4. **Fatal Collision:** Core 0 has **not yet finished saving its registers to Thread A's stack!** Core 0 and Core 1 push, pop, and mutate the exact same stack frame simultaneously.

3.2 The Deferred Publish / Claim Mechanism

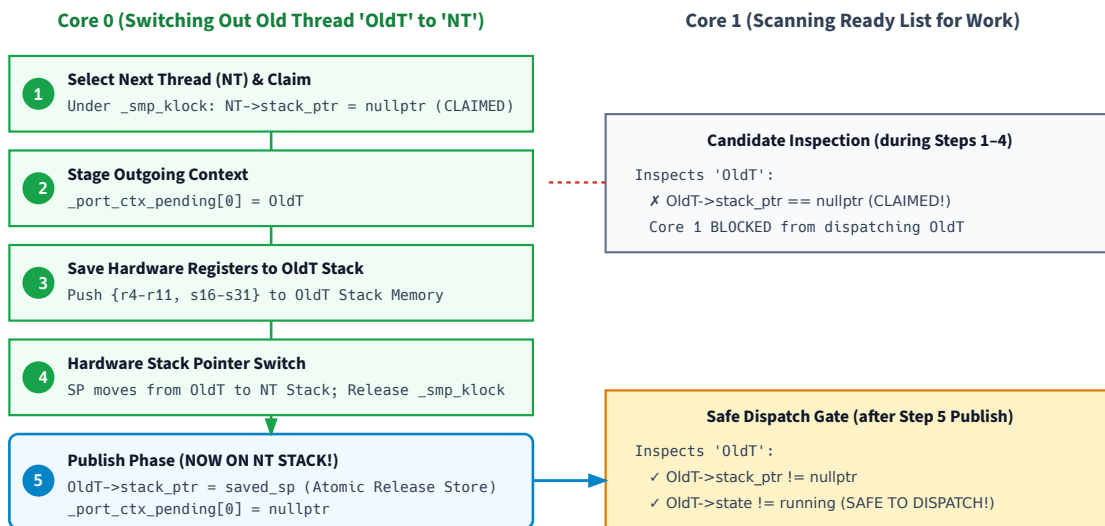


Figure 4: Figure 4: Two-Phase Deferred Publish / Claim Protocol during Multi-Core Context Switching

1. **Claim Phase:** In `scheduler::internal_switch_threads()`, `next_thread->context_.port_.stack_ptr = nullptr`. This marks the candidate thread as in-flight.
2. **Deferred Stage:** The outgoing thread pointer is saved in `_port_ctx_pending[port_cpu_id()]`.
3. **Register Commit:** Hardware registers are pushed onto the old stack, and the CPU stack pointer switches to the new stack.
4. **Publish Phase:** Executing **on the new thread's stack**, the core publishes the saved stack pointer:

```
void publish_pending (void) {
    unsigned cpu = port_cpu_id ();
    rtos::thread* old = (rtos::thread*) _port_ctx_pending[cpu];
    if (old != nullptr) {
        _port_ctx_pending[cpu] = 0;
        __atomic_store_n (&old->context_.port_.stack_ptr, old_saved_sp, __ATOMIC_RELEASE);
    }
}
```

5. **Scheduler Gate:** The ready list picker will only dispatch candidate thread `th` if:

```
(th == old_thread) || (th->state_ != rtos::thread::state::running && th-
>context_.port_.stack_ptr != nullptr)
```

3.3 Floating Point Unit (FPU) Context & Lazy Stacking

On ARM Cortex-M4F/M7F/M33 architectures, floating point registers `{s0-s31, FPSCR}` consume 136 bytes of stack.

The `smp` port configures hardware **Lazy Stacking** (`FPCCR.ASPEN = 1`, `FPCCR.LSPEN = 1`):

- Exception entry reserves space on stack without copying; actual register stacking is deferred until an FPU instruction executes inside the handler.
- In `PendSV_Handler`, the kernel inspects `EXC_RETURN` bit 4 to save/restore `{s16-s31}` only for threads that actually used floating-point instructions.

4 Scheduler Architecture & Thread Lifecycle

4.1 Scheduler Data Structures

```
// Single-Core (xpack-development)
namespace os::rtos::scheduler {
    extern thread* current_thread_;           // Scalar pointer to single active thread
    extern thread* os_idle_thread_;          // Single idle thread
    extern volatile state_t lock_state;       // Scalar lock state
}

// Multi-Core SMP (smp)
namespace os::rtos::scheduler {
    extern thread* current_thread_[OS_NCPU]; // Array of active threads per core
    extern thread* os_idle_thread_core[OS_NCPU]; // Array of idle threads per core
    extern volatile state_t lock_state[OS_NCPU]; // Array of lock states per core
    extern volatile unsigned _port_ctx_pending[OS_NCPU]; // Array of pending publish
    pointers
}
```

4.2 Thread CPU Affinity & The Multi-Core Ready-List Picker

Threads carry a 32-bit CPU affinity mask (`th_cpu_affinity`). During context switch on CPU Core *C*:

```
rtos::thread* internal_switch_threads (void) {
    const unsigned cpu = port_cpu_id ();
    rtos::thread* old_thread = current_thread_[cpu];

    for (auto&& th : ready_list_) {
        // 1. Check affinity mask
        if ((th.cpu_affinity () & (1u << cpu)) == 0) continue;
        // 2. Check if unclaimed and not active on another core
        if (is_thread_ready_to_run (&th, old_thread)) {
            th.context_.port_.stack_ptr = nullptr; // Claim
            current_thread_[cpu] = &th;
            return &th;
        }
    }
    // 3. Fallback to THIS core's idle thread
    current_thread_[cpu] = os_idle_thread_core[cpu];
    return os_idle_thread_core[cpu];
}
```

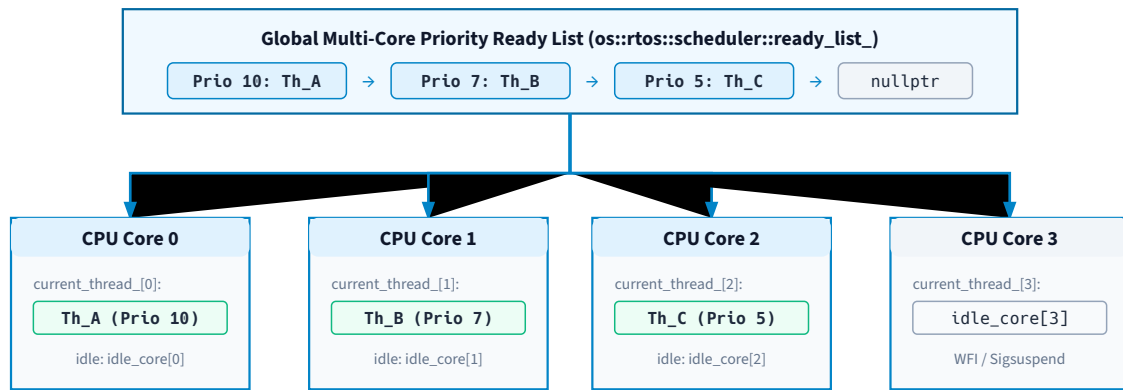


Figure 5: Figure 5: SMP Ready List Dispatch Topology Across 4 Symmetrical CPU Cores

4.3 Thread Lifecycle, Teardown & `join()` Synchronization

4.3.1 1. Arbitration via `state::destroying = 7`

To prevent a double-free race between `thread::kill()` on Core 1 and the idle thread reaper on Core 0, an intermediate state `state::destroying = 7` is introduced. The first core to transition `state_` under `_smp_klock` claims exclusive destruction ownership.

4.3.2 2. SMP `join()` Spin-Wait

In `thread::join()`, the calling core waits until the terminating thread is completely off all CPU cores (`scheduler::current_thread[c] != this`) before returning, ensuring user code cannot free stack memory while an outgoing core is still committing registers to it.

4.4 Inter-Processor Interrupts (IPI)

When Core 0 unblocks high-priority Thread H whose affinity pins it to Core 1, Core 0 invokes `port_smp_ipi(1)`. Core 1 receives the hardware IPI, enters its context switch handler, and dispatches Thread H immediately, avoiding up to a 1 ms tick latency.

5 Intrusive Lists & Core Data Structures

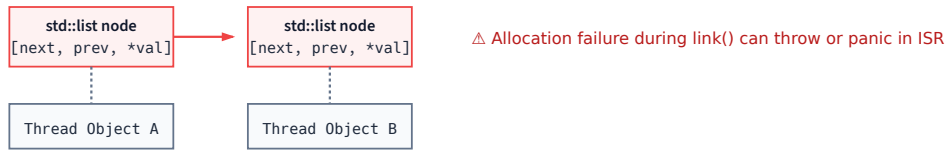
5.1 Why Intrusive Containers in Embedded RTOS Design

Standard containers (`std::list<T>`) dynamically allocate wrapper nodes on the heap during insertion. In an RTOS:

- Heap allocation introduces non-deterministic execution times.
- Out-of-memory errors during thread blocking or signaling are unacceptable.
- Dynamic nodes introduce cache misses.

µOS++ employs **Intrusive Doubly-Linked Lists** (`os::utils::double_list`). The node links (`next_`, `prev_`) reside **directly inside the managed objects** (`rtos::thread`, `rtos::mutex`, `rtos::timer_node`).

Non-Intrusive Container (`std::list<Thread>`) — Extra Dynamic Heap Allocations



Intrusive Container (`os::utils::double_list`) — Zero Dynamic Memory Allocations



Figure 6: Figure 6: Memory Layout Comparison — Non-Intrusive (`std::list`) vs. Embedded Intrusive (`double_list`)

- **Zero Dynamic Allocation:** Linking a thread into the ready list requires zero heap operations.
- **Guaranteed Success:** Insertion can never fail due to memory exhaustion.
- **$O(1)$ Removal:** Any object unlinks itself in $O(1)$ time via `node->unlink()`.

5.2 Static Destruction & The Clean `_Exit()` Bypass

The destructor `double_list::~~double_list()` asserts `empty()`. In multi-core test environments, calling `std::exit(0)` invokes static destructors while secondary cores are still executing idle threads. The `smp` test framework terminates via `std::_Exit(0)` (or semihosting `SYS_EXIT`), bypassing static destructors and cleanly halting hardware cores.

6 C++ Object-Oriented Solutions, Idioms & Standards

1. **RAII Scope Guards:** `interrupts::critical_section` automatically masks IRQs and acquires `_smp_klock` on construction; releases `_smp_klock` and restores prior IRQ state on destruction.
2. **Polymorphic Memory Resources (`pmr`):** Fixed-size deterministic pool allocators (`block_pool`), general-purpose heaps (`first_fit_top`), and stack allocators (`lifo`). Arithmetic overflow checks were added to `align_size()` and `malloc.cpp:calloc()`.
3. **CMSIS-RTOS Dual-Layer Architecture:** Clean modern C++ classes wrapped with standard C CMSIS-RTOS APIs. Polymorphic deletion bugs were eliminated by checking runtime type tags and deleting through concrete derived types.
4. **C++20 Volatile Deprecation:** Compound operations on volatile types (`volatile int x; ++x;`) across device drivers (`usb_dwc2.cpp`) and test suites were upgraded to `std::atomic<T>` with explicit memory order parameters.

7 Architecture-Specific Ports: Deep Technical Breakdown

POSIX Host	Cortex-M (RP2350)	ARMv7-A / A32	ARMv8-A / A64	RISC-V (RV32/64)
CPU MODEL 1 pthread = 1 CPU	CPU MODEL 2x Cortex-M33	CPU MODEL BCM2837 / RK3506	CPU MODEL 64-bit BCM2837	CPU MODEL Hart ID (mhartid)
INTERRUPTS pthread_sigmask	INTERRUPTS PRIMASK / BASEPRI	INTERRUPTS CPSID / CPSIE	INTERRUPTS MSR DAIFSET/CLR	INTERRUPTS CSR mstatus.MIE
TICK TIMER timer_create TID	LOCK MECHANISM SIO Spinlock 0	LOCK MECHANISM LDREX / STREX	LOCK MECHANISM LDAXR / STLXR	LOCK MECHANISM AMO.SWAP / LR/SC
IPI SIGNAL pthread_kill	IPI SIGNAL SIO FIFO IRQ 25	IPI SIGNAL GIC SGI / Mailbox	IPI SIGNAL GIC SGI / Mailbox	IPI SIGNAL CLINT (msip)
CONTEXT ucontext_t	CONTEXT PendSV + Lazy FP	CONTEXT Software Frame	CONTEXT 64-bit x0-x30, Q	CONTEXT ra, sp, CSRs

Figure 7: Figure 7: Architecture Port Hardware Mapping and Synchronization Comparison Matrix

7.1 POSIX Native Host Port (`micro-os-plus-iii-posix-arch`)

- **Machine Model:** One host `pthread` represents one physical CPU core. N host threads (`g_cpu_thread[0S_NCPU]`) are spawned at boot.
- **Interrupt Masking:** `pthread_sigmask(SIG_BLOCK, &irq_set, &old)` masks signals per host thread, mirroring per-CPU interrupt masking.
- **Per-CPU Tick:** `timer_create` with `SIGEV_THREAD_ID` delivers `SIGRTMIN` ticks to each host thread individually.
- **IPI:** `pthread_kill(g_cpu_thread[target_cpu], SIGRTMIN + 1)`.
- **The Clang TLS Caching Trap:** Clang 16–18 at `-O2` assumes thread pointer `%fs:0` is constant across a function. If a μOS++ thread blocks and resumes on a different host thread (migration), the cached register returns the **wrong CPU ID**. The accessor `port_cpu_id()` is compiled out-of-line with a compiler memory barrier (`asm volatile (" ::: \"memory\")`) to force `%fs` re-reading.

7.2 ARM Cortex-M Port (`micro-os-plus-iii-cortexm`)

7.2.1 1. Raspberry Pi Pico 2 (RP2350 Dual-Core Cortex-M33)

- **Kernel Lock:** Implemented via **RP2350 Hardware SIO Spinlock 0** (`0xD0000100`). Standard `LDREX / STREX` does not work across cores because the RP2350 lacks an inter-core global exclusive monitor.
- **IPI:** Uses SIO Inter-Core FIFO IRQ 25 (`SIO_IRQ_FIFO`).
- **Core 1 Boot:** Launched via Bootrom FIFO handshake (`launch_core1()`).
- **High-Res Clock:** 64-bit latched hardware timer (`TIMER0 TIMEHR / TIMELR`) at 1 MHz.

7.2.2 2. ARM Generic Dual-Core (SSE-200)

- Emulated in QEMU (`mps2-an505 / mps2-an521`) using ARM Message Handling Unit (MHU) and `CPUID` registers.

7.3 ARMv7-A / AArch32 Port (`micro-os-plus-iii-aarch32`)

- **Hardware Targets:** Broadcom BCM2837 (Raspberry Pi Zero 2 W / 3B, 4 cores) and Rockchip RK3506 (Luckfox Lyra, 3 cores).
- **Core Identification:** Reads CP15 MPIDR register (`mrc p15, 0, Rd, c0, 5`) and masks Affinity Level 0 (`& 0xFFu`).
- **Interrupts:** ARM GIC-400 SGIs on RK3506; Broadcom local mailboxes on BCM2837.
- **MMU:** Short-descriptor 1 MB section translation tables with Inner Shareable Normal Cacheable DRAM attributes.

7.4 ARMv8-A / AArch64 Port (`micro-os-plus-iii-aarch64`)

- **Execution State:** 64-bit general-purpose registers `x0-x30`, `SP_EL0`, `SPSR_EL1`.
- **Interrupt Masking:** `msr daifset, #2` (disable IRQ) / `msr daifclr, #2` (enable IRQ).

- **Semihosting & Fault Handling:** `HLT 0xF000` traps. Implements an independent UART-only `FaultConsole` to avoid fatal recursive semihosting traps when a JTAG debugger is disconnected during a panic.

7.5 RISC-V Port (`micro-os-plus-iii-riscv`)

- **Hart Model:** Extracts hardware thread index via `csrr a0, mhartid`.
- **Interrupt Control:** Manipulates `mstatus.MIE` and Core Local Interruptor (CLINT) memory-mapped registers (`msip` for software IPIs, `mtime` / `mtimecmp` for hardware ticks).
- **Atomics:** Utilizes RISC-V atomic memory instructions (`AMOADD.W`, `AMOSWAP.W.AQRL`, `LR.W` / `SC.W`).

8 Hardening & Upstream Integration Strategy

8.1 Summary of Fixed Defects

Component	Defect Description	SMP Impact	Resolution
<code>os-condvar.cpp</code>	<code>wait()</code> unlocked/locked without suspending.	Cross-core lock thrashing & bus saturation.	Implemented full suspend, link, unlock, reschedule, and re-acquire sequence.
<code>os-mutex.cpp</code>	<code>boosted_prio_</code> overwritten by lower waiter.	Unbounded priority inversion across cores.	Track maximum waiter priority across all held mutexes.
<code>os-thread.cpp</code>	<code>join()</code> returned before target was off stack.	Stack corruption on multi-core.	Added spin-wait checking <code>current_thread[c] != this</code> across all cores.
<code>os-memory.h</code>	<code>align_size()</code> overflowed <code>SIZE_MAX</code> .	Heap corruption on large requests.	Added overflow saturation checks.
<code>file-descriptors-manager.cpp</code>	Unprotected static file descriptor array.	Duplicate file descriptors and leaked handles.	Wrapped operations in <code>interrupts::critical_section</code> .

8.2 The Inviolable Migration Rules

1. **Granular Code Chunks, Not File Copies:** A step consists of small, cohesive code snippets.
2. **Cross-Repository Synchronization:** Kernel and port packages are updated and linked in the exact same step.
3. **Logical Progression:** Part A (Steps 1–13: single-core fixes), Part B (Steps 14–23: SMP primitives behind `#if defined(OS_USE_SMP_SCHEDULER)`), Part C (Steps 24–28: port releases and add-only test extensions), Part D (Steps 29–30: docs and final merge).
4. **Pristine Test Procedures:** Existing test suites, platforms, and `package.json` configurations are never modified.
5. **Continuous 72/72 Test Gate:** Every step must pass all 24 toolchain configurations under `-Werror`.

8.3 Automated Step Verification Script (`scripts/verify-step.sh`)

```
#!/usr/bin/env bash
# Automated Step Verification Gate: checks undef invariant, links ports, runs 72 tests
set -euo pipefail
STEP_NUM="${1:-}"
[ -z "$STEP_NUM" ] && { echo "Usage: $0 <step>"; exit 1; }

# Check undef invariant for Part B (zero single-core delta)
if [ "$STEP_NUM" -ge 14 ] && [ "$STEP_NUM" -le 23 ]; then
    for f in $(git diff --name-only origin/xpack-development HEAD -- include/ src/); do
        git show origin/xpack-development:"$f" | undef -UOS_USE_SMP_SCHEDULER > /tmp/old_sc
    || true
        git show HEAD:"$f" | undef -UOS_USE_SMP_SCHEDULER > /tmp/new_sc || true
    done
fi
```

```
diff -u /tmp/old_sc /tmp/new_sc > /dev/null || { echo "Divergence in $f"; exit 2; }
done
fi

# Link local ports across all 24 configurations and run test suite
cd ../micro-os-plus-iii-posix-arch && xpm link
cd ../micro-os-plus-iii-cortexm && xpm link
cd ../micro-os-plus-iii/tests
xpm run test-all # 72 / 72 Pass Gate
```

8.4 Comprehensive 30-Step Execution Checklist

Step	Phase	Core Action / Snippet	Kernel (K)	Cortex-M (C)	POSIX-Arch (P)	Gate Check
01	Part A	ISO C DIR, Glibc timegm(), Newlib types, List iterators	posix/dirent.h, timegm.c	—	—	72/72 Pass
02	Part A	align_size wrap check, heap usable size, calloc overflow	os-memory.cpp, first-fit-top.cpp	—	—	72/72 Pass
03	Part A	C++17 operator new(align_val_t), static system_error, chrono	new.cpp, system-error.cpp	—	—	72/72 Pass
04	Part A	One-shot timer default, polymorphic deletion, 64-bit timeouts	os-c-wrapper.cpp	—	—	72/72 Pass
05	Part A	File descriptor manager mutexing, block device size fix	file-descriptors-manager.cpp	—	—	72/72 Pass
06	Part A	ARMv8-M mainline macros, semihosting traps, UART mirror	semihosting.h, exception-handlers.c	—	—	72/72 Pass
07	Part A	Timer callback outside critical section, ISR errno	os-lists.cpp, os-timer.cpp	—	—	72/72 Pass
08	Part A	Mutex priority inheritance and ceiling fix	os-mutex.cpp	—	—	72/72 Pass
09	Part A	Thread state destroying (7), atomic join, detach	os-thread.cpp, os-idle.cpp	—	—	72/72 Pass
10	Part A	CondVar atomic list insert + unlock, high-precision timeout	os-condvar.cpp	—	—	72/72 Pass
11	Part A	std::thread::join synchronization, functor lifetime	thread-cpp.h	—	—	72/72 Pass
12	Part A	Message queue reschedule yield triggers	os-mqueue.cpp	—	—	72/72 Pass

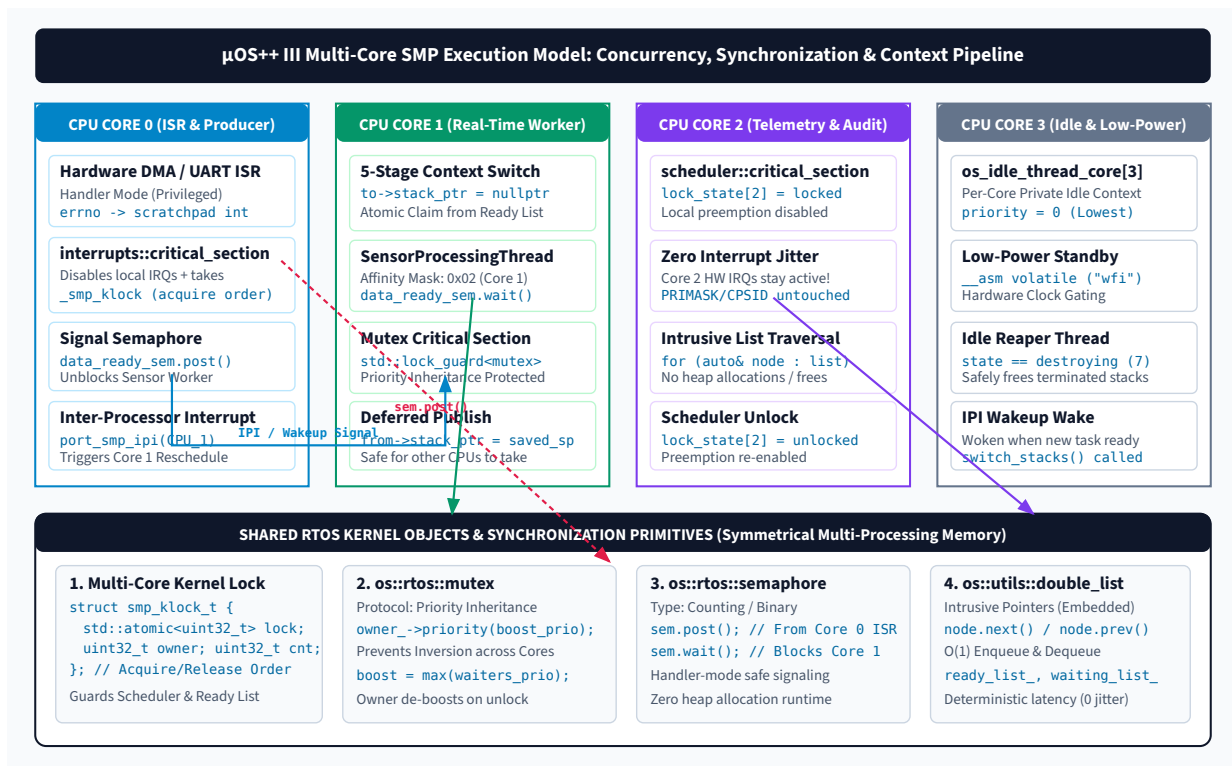
13	Part A	<code>clock_highres</code> hardware counter port synchronization	<code>os-clocks.cpp</code>	<code>os-inlines.h</code>	<code>os-inlines.h</code>	72/72 Pass
14	Part B	<code>port_cpu_id()</code> , POSIX-arch compiler barrier	<code>os-core.cpp</code>	<code>os-inlines.h</code>	<code>os-core.cpp</code>	Unifdef + 72/72
15	Part B	Per-CPU scheduler lock state <code>lock_state[OS_NCPU]</code>	—	<code>os-decls.h</code>	<code>os-core.cpp</code>	Unifdef + 72/72
16	Part B	Multi-core recursive kernel lock <code>_smp_klock</code>	<code>os-c-decls.h</code>	<code>os-decls.h</code>	<code>os-core.cpp</code>	Unifdef + 72/72
17	Part B	Per-CPU interrupt tracking (<code>_in_isr[OS_NCPU]</code> , <code>irq_set</code>)	—	—	<code>os-decls.h</code>	Unifdef + 72/72
18	Part B	Per-CPU current thread pointer <code>current_thread_[OS_NCPU]</code>	<code>os-sched.h</code>	<code>os-core.cpp</code>	<code>os-core.cpp</code>	Unifdef + 72/72
19	Part B	Per-CPU idle thread instances <code>os_idle_thread_core</code>	<code>os-core.cpp</code>	<code>os-core.cpp</code>	<code>os-core.cpp</code>	Unifdef + 72/72
20	Part B	Thread CPU affinity masking (<code>th_cpu_affinity</code>)	<code>os-thread.cpp</code>	—	—	Unifdef + 72/72
21	Part B	5-stage deferred publish/claim context switch (<code>stack_ptr</code>)	<code>os-idle.cpp</code>	<code>os-core.cpp</code>	<code>os-core.cpp</code>	Unifdef + 72/72
22	Part B	Multi-core ready list thread picker	<code>os-core.cpp</code>	—	—	Unifdef + 72/72
23	Part B	POSIX-arch host thread per CPU, IPI signal engine	<code>os-thread.cpp</code>	—	<code>host_cpu.cpp</code>	Smoke + 72/72
24	Part C	Tag and release ports (<code>posix-arch v1.1.0</code> , <code>cortexm v1.2.0</code>)	—	Release v1.2.0	Release v1.1.0	72/72 Pass
25	Part C	New architecture files (M33, RP2350 spinlocks)	<code>os-thread.cpp</code>	<code>os-core-m33.cpp</code>	<code>board-contract.cpp</code>	72/72 Pass
26	Part C	Modular CMake build scripts and targets	<code>cmake/</code>	—	—	72/72 Pass
27	Part C	New test sources (<code>fp-switch</code> , <code>smp-support</code> , linker scripts)	<code>tests/sources/</code>	—	—	72/72 Pass
28	Part C	Add-only test platforms (<code>2xcortex-m33</code> , <code>pico2</code> , <code>aarch32/64</code>)	<code>tests/platforms/</code>	—	—	72 Old + New Pass

29	Part D	Complete technical documentation & Typst PDFs	docs/	—	—	Clean Doc Build
30	Part D	Final branch alignment, repository restore, upstream PR	All Repos	All Repos	All Repos	Full Pass

9 Holistic Architecture in Action: Complete Logical Multi-Core Example

9.1 Architectural Scenario & Execution Model

To demonstrate how all the architectural components—SMP threads, thread CPU affinity, recursive kernel locks (`_smp_klock`), critical sections (`scheduler` vs. `interrupts`), hardware ISR interactions, 5-stage context switching, mutex priority inheritance, semaphores, and intrusive lists—operate concurrently in real hardware, this section analyzes a complete embedded application running across four symmetrical CPU cores.



9.2 Complete C++ Application Code

The following listing is a complete, standards-compliant μOS++ III C++ application executing across 4 symmetrical cores:

```
#include <cmsis-plus/rtos/os.h>
#include <cmsis-plus/utils/lists.h>
#include <array>
#include <cstdint>
#include <cstdio>

using namespace os::rtos;

// 1. Intrusive Data Structure (Zero Heap Overhead during Runtime)
struct telemetry_sample_t : public os::utils::double_list_links {
    uint32_t timestamp;
    uint32_t sensor_id;
    float    reading;
```

```

};
static os::utils::double_list g_telemetry_queue;

// 2. Shared Kernel Synchronization Primitives
static semaphore_counting g_data_ready_sem{ 0, 100 };
static mutex g_shared_buffer_mutex{ mutex::initializer_recursive };

static constexpr size_t POOL_SIZE = 16;
static std::array<telemetry_sample_t, POOL_SIZE> g_sample_pool;
static size_t g_pool_index = 0;

static constexpr size_t RING_BUFFER_SIZE = 32;
static uint32_t g_raw_ring_buffer[RING_BUFFER_SIZE];
static volatile size_t g_ring_head = 0;
static volatile size_t g_ring_tail = 0;

// 3. Hardware ISR Simulation (Executes on Core 0 in Handler Mode)
extern "C" void dma_uart_hardware_isr (void) {
    // Enter kernel critical section (disables local IRQ, takes _smp_klock with acquire
    // order)
    {
        interrupts::critical_section ics;
        uint32_t raw_data = 0xABCD1234;
        size_t next_head = (g_ring_head + 1) % RING_BUFFER_SIZE;
        if (next_head != g_ring_tail) {
            g_raw_ring_buffer[g_ring_head] = raw_data;
            g_ring_head = next_head;
        }
    } // _smp_klock released with release order, local IRQ restored

    // Post semaphore to unblock consumer thread on Core 1
    g_data_ready_sem.post ();
}

// 4. Real-Time Consumer Thread (Pinned to CPU Core 1)
static void* sensor_processing_thread_func (void* args) {
    (void)args;
    while (true) {
        result_t res = g_data_ready_sem.wait (); // Atomically sleeps, 5-stage switch
        if (res != result::ok) continue;

        uint32_t extracted_data = 0;
        // Mutex with Priority Inheritance: boosts lower-priority owner if contended
        {
            std::lock_guard<mutex> lock (g_shared_buffer_mutex);
            if (g_ring_tail != g_ring_head) {
                extracted_data = g_raw_ring_buffer[g_ring_tail];
                g_ring_tail = (g_ring_tail + 1) % RING_BUFFER_SIZE;
            }
        }

        // Enqueue onto intrusive list under critical section
        {
            interrupts::critical_section ics;
            telemetry_sample_t* sample = &g_sample_pool[g_pool_index++ % POOL_SIZE];
            sample->timestamp = static_cast<uint32_t> (clock_systick::now ());
            sample->sensor_id = 1;
            sample->reading = static_cast<float> (extracted_data & 0xFFFF) * 0.01f;
            g_telemetry_queue.link_tail (*sample); // 0(1) intrusive enqueue, zero heap
        }
    }
}

```

```

allocation
    }
}
return nullptr;
}

// 5. Telemetry & Audit Thread (Pinned to CPU Core 2)
static void* telemetry_worker_thread_func (void* args) {
    (void)args;
    while (true) {
        clock_systick::sleep_for (100);

        // Scheduler Critical Section: lock_state[Core2] = locked.
        // Local preemption disabled, BUT Core 2 hardware interrupts remain active (Zero
        Jitter)!
        {
            scheduler::critical_section scs;
            size_t count = 0;
            for (auto& node : g_telemetry_queue) {
                telemetry_sample_t& sample = static_cast<telemetry_sample_t&> (node);
                count++;
            }
            trace::printf ("[Core %u] Telemetry sweep processed %zu items.\n", port_cpu_id (),
count);
        } // Preemption re-enabled on Core 2
    }
    return nullptr;
}

// 6. Application Initialization & Core Startup (Main Thread on Core 0)
int os_main (int argc, char* argv[]) {
    thread::attributes sensor_attr = thread::initializer;
    sensor_attr.th_priority = thread::priority::high;
#ifdef OS_USE_SMP_SCHEDULER
    sensor_attr.th_cpu_affinity = (1U << 1); // Pinned to Core 1
#endif
    thread sensor_thread{ "sensor-consumer", sensor_processing_thread_func, nullptr,
sensor_attr };

    thread::attributes telemetry_attr = thread::initializer;
    telemetry_attr.th_priority = thread::priority::normal;
#ifdef OS_USE_SMP_SCHEDULER
    telemetry_attr.th_cpu_affinity = (1U << 2); // Pinned to Core 2
#endif
    thread telemetry_thread{ "telemetry-audit", telemetry_worker_thread_func, nullptr,
telemetry_attr };

    for (size_t i = 0; i < 5; ++i) {
        clock_systick::sleep_for (50);
        dma_uart_hardware_isr ();
    }
    return 0;
}

```

9.3 Step-by-Step State Transition Walkthrough

1. **Core 0 (ISR & Producer):** Executes in privileged Handler Mode. `this_thread::__errno()` routes to a scratch-pad integer. `interrupts::critical_section` masks local IRQs and takes `_smp_klock` with acquire ordering. Posting `g_data_ready_sem` puts `SensorProcessingThread` into the ready list and triggers `port_smp_ipi(1)`.

2. **Core 1 (5-Stage Context Switch & Consumer):** Core 1 receives the IPI and invokes `switch_stacks()` :
 - **Stage 1 (Atomic Claim):** `to->stack_ptr = nullptr` . Locked against Cores 2 and 3.
 - **Stage 2 (Register Spill):** Saves callee registers (R4–R11) to idle thread stack.
 - **Stage 3 (SP Switch):** Hardware `SP` register loaded with `SensorProcessingThread->sp` .
 - **Stage 4 (Deferred Publish):** `os_idle_thread_core[1]->stack_ptr = saved_sp` . Only now is the idle thread published as restorable.
 - **Stage 5 (Register Restore):** Registers popped and `SensorProcessingThread` resumes execution.
3. **Core 1 (Mutex Priority Inheritance):** Takes `g_shared_buffer_mutex` . If unowned, ownership is granted immediately. If owned by a lower-priority task, the owner's priority is boosted across cores until unlocked, preventing priority inversion.
4. **Core 2 (Scheduler Critical Section & Zero Interrupt Jitter):** Enters `scheduler::critical_section` . Sets `lock_state[Core2] = locked` . Preemption on Core 2 is disabled, but **Core 2 hardware interrupts remain active** (zero interrupt jitter). It iterates through intrusive `g_telemetry_queue` nodes via `node.next()` with zero dynamic heap overhead.
5. **Core 3 (Idle Standby & Garbage Collection Reaper):** Runs `os_idle_thread_core[3]` in a low-power `wfi` loop. Woken by IPI when new tasks are dispatched. The idle reaper safely destroys finished threads that are in State 7 (`destroying`) and verified off-stack (`stack_ptr == nullptr`).

9.4 Single-Core vs. Multi-Core SMP Behavioral Comparison

Architectural Dimension	Uniprocessor (<code>xpack-development</code>)	Multi-Core SMP (<code>smp</code>)
Thread Concurrency	Time-sliced interleaving on CPU 0. One thread active at any instant.	True parallel hardware execution across Cores 0, 1, 2, and 3 simultaneously.
ISR Preemption	ISR preempts active thread on CPU 0; worker thread waits until ISR exits.	ISR runs on Core 0; consumer thread immediately executes concurrently on Core 1 via IPI.
Interrupt Jitter	<code>interrupts::critical_section</code> disables CPU interrupts globally.	<code>scheduler::critical_section</code> disables preemption on Core 2 while Core 2 IRQs stay 100% active.
Synchronization	Simple interrupt masking (<code>cpsid i</code>). Zero spinlocks.	Recursive spinlock (<code>_smp_klock</code>) with acquire/release memory semantics.
Context Switch	Synchronous stack push/pop; no multi-core hazard.	5-Stage Deferred Publish / Claim (<code>stack_ptr == nullptr</code> claim + deferred publish).
Data Structures	Intrusive <code>double_list</code> (zero heap fragmentation).	Same zero-heap intrusive efficiency, guarded by SMP spinlocks across cores.

— End of Technical Document —